

|                 |                                                 |
|-----------------|-------------------------------------------------|
| Document number | P0805R2                                         |
| Date            | 2018-06-22                                      |
| Project         | Programming Language C++, Library Working Group |
| Reply-to        | Marshall Clow <mclow.lists@gmail.com>           |

---

# Comparing Containers

## History

[P0805R0](#) was presented to LEWG in Albuquerque, and was received favorably. They asked for an R1 that also included the rest of the sequence containers (`<deque>`, `<list>`, `<forward_list>` and `<vector>`) and forwarded the paper to LWG.

In Rapperswil, LWG asked for a couple of editorial changes to [P0805R1](#) (reordering template parameters and function lists), and I rebased on N4750. Also, Daniel pointed out that I was relying on Table 77 and 78 for semantics of the comparison operators, and they only define semantics for comparisons between containers of the same type.

For [P0805R2](#) Daniel, Jonathan and Casey helped me add the appropriate wording for comparing containers of different types.

## Rationale

When I was attempting to implement [P0122R5](#), I became frustrated with the inability to compare spans of different lengths, and sometimes spans of the same lengths as well. I started thinking about this in general, and realized that our comparison conventions for containers are limited, but for no good reason.

In C++17, `std::optional` has a nice set of comparison operators.

- An `optional<T>` can be compared to a `T`.
- An `optional<T>` can be compared to a `U`.
- An `optional<T>` can be compared to an `optional<T>`.
- An `optional<T>` can be compared to an `optional<U>`.
- An `optional<T>` can be compared to a `nullopt_t`.

But an `array<int, 5>` cannot be compared to an `array<int, 6>`. Nor can an `array<int, 5>` cannot be compared to an `array<long, 5>`. This seems like an artificial limitation to me - these comparisons "make sense", we just don't implement them. We *can* compare a `vector<int>` that contains five elements to a `vector<int>` that contains six elements, but not a `vector<int>` to a `vector<long>`, no matter what size. And if the allocators are different, that won't work either.

As people write more and more generic code, these arbitrary limitations will cause increasing friction.

In the header `<algorithm>` we have `lexicographical_compare`. This is a very general algorithm - it compares two sequences, which may be of differing lengths, and/or different underlying types, and returns whether one is "less than" the other. But that generality is not reflected in the containers, even though their comparisons are defined in terms of `lexicographical_compare` (Table 85).

The same logic applies to `std::tuple`. The comparison operators for `tuple` require that both tuples be the same size (`[tuple.rel]/1`) but (surprisingly) not that they contain the same types.

Note: I have not suggested changes to `basic_string`, because it does not use `lexicographical_compare`, but delegates to a traits class.

## Suggested changes

These wording changes are relative to [N4750](#).

In `[tuple.rel]/1`, change:

*Requires:* For all `i`, where  $0 \leq i$  and  $i < \min(\text{sizeof}...(TTypes), \text{sizeof}...(UTypes))$ , ~~`sizeof...(TTypes)`~~, `get<i>(t) == get<i>(u)` is a valid expression ~~returning a type~~ that is convertible to `bool`. ~~`sizeof...(TTypes) == sizeof...(UTypes)`~~;

In `[tuple.rel]/2`, change:

*Returns:* `true` if `sizeof...(TTypes) == sizeof...(UTypes)` and `get<i>(t) == get<i>(u)` for all `i`, otherwise `false`. For any two zero-length tuples `e` and `f`, `e == f` returns `true`.

In `[tuple.rel]/5`, change:

*Requires:* For all `i`, where  $0 \leq i$  and  $i < \min(\text{sizeof}...(TTypes), \text{sizeof}...(UTypes))$ , ~~`i < sizeof...(TTypes)`~~, both `get<i>(t) < get<i>(u)` and `get<i>(u) < get<i>(t)` are valid expressions ~~returning types~~ that are convertible to `bool`. ~~`sizeof...(TTypes) == sizeof...(UTypes)`~~;

In `[tuple.rel]/6`, change:

*Returns:* The result of a lexicographical comparison between `t` and `u`. A zero-length tuple is lexicographically less than any non-zero-length tuple. The result is defined as: `(bool)(get<0>(t) < get<0>(u)) || (!(bool)(get<0>(u) < get<0>(t)) && ttail < utail)`, where `rtail` for some tuple `r` is a tuple containing all but the first element of `r`. For any two zero-length tuples `e` and `f`, `e < f` returns `false`.

In `[sequence.reqmts]/3`, change:

In Tables 81 and 82, `x` denotes a sequence container class containing elements of type `T`, `a` denotes a value of type `X`, `x2` denotes a different specialization of the same class template as `x` (if any) containing elements of type `T2`, `a2` denotes a value of type `x2`, ~~`a` denotes a value of type `x` containing elements of type `T`~~, `u` denotes the name of a variable being declared [...]

Append six rows to the end of table 81:

| Expression              | Return type                             | Assertion/note pre-/post-condition                                                                                                                                                                                                                                       |
|-------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>a == a2</code>    | <u>convertible to <code>bool</code></u> | <u><i>Returns:</i> <code>equal(a.begin(), a.end(), a2.begin(), a2.end())</code>. <i>Requires:</i> <code>T</code> and <code>T2</code> model <code>EqualityComparableWith</code>. <i>Complexity:</i> Constant if <code>a.size() != a2.size()</code>, linear otherwise.</u> |
| <code>a != a2</code>    | <u>convertible to <code>bool</code></u> | <u>Equivalent to <code>!(a == a2)</code>.</u>                                                                                                                                                                                                                            |
| <code>a &lt; a2</code>  | <u>convertible to <code>bool</code></u> | <u><i>Returns:</i> <code>lexicographical_compare(a.begin(), a.end(), a2.begin(), a2.end())</code>. <i>Requires:</i> <code>T</code> and <code>T2</code> model <code>StrictTotallyOrdered</code>. <i>Complexity:</i> linear.</u>                                           |
| <code>a &gt; a2</code>  | <u>convertible to <code>bool</code></u> | <u>Equivalent to <code>a2 &lt; a</code></u>                                                                                                                                                                                                                              |
| <code>a &lt;= a2</code> | <u>convertible to <code>bool</code></u> | <u>Equivalent to <code>!(a &gt; a2)</code>.</u>                                                                                                                                                                                                                          |
| <code>a &gt;= a2</code> | <u>convertible to <code>bool</code></u> | <u>Equivalent to <code>!(a &lt; a2)</code>.</u>                                                                                                                                                                                                                          |



```
bool operator>=(const list<T1, Allocator1>& x, const list<T2, Allocator2>& y);
```

In [vector.syn], add the following functions:

```
template<class T1, class Allocator1, class T2, class Allocator2>
    bool operator==(const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);
template<class T1, class Allocator1, class T2, class Allocator2>
    bool operator!=(const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);
template<class T1, class Allocator1, class T2, class Allocator2>
    bool operator< (const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);
template<class T1, class Allocator1, class T2, class Allocator2>
    bool operator> (const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);
template<class T1, class Allocator1, class T2, class Allocator2>
    bool operator<=(const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);
template<class T1, class Allocator1, class T2, class Allocator2>
    bool operator>=(const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);
```

## Implementation status

This has been implemented (but not shipped) for libc++.

## Future Directions

## Thanks

Thanks to Daniel Krügler for his advice and wording assistance. Thanks to Jonathan Wakely and Casey Carter for their wording assistance.